

```

1535         -float(timeline["energy_kwh"]),
1536     )
1537     if best is None or score > best[0]:
1538         best = (score, extra_ids)
1539     if best is not None:
1540         chosen_soft = best[1]
1541         soft = soft[~soft["货箱编号"].astype(str).isin(chosen_soft)]
1542     hard_specs.append(
1543         {"stops": [{"service_id": service_id, "box_ids": hard_ids + chosen_soft}],
1544          "kind": "hard", **({"forced_unit": forced_unit} if forced_unit else {})}
1545     )
1546     if len(soft):
1547         for batch in exact_q1_service(
1548             data["dem"], data["nodes"]["001"], data["nodes"][service_id], soft,
1549             data["transport_types"],
1550         ):
1551             soft_specs.append(
1552                 {"stops": [{"service_id": service_id, "box_ids": list(batch["box_ids"])}],
1553                  "kind": "soft"}
1554             )
1555
1556 # Pair compatible soft single-service batches when the merged route saves energy.
1557 active = soft_specs[:]
1558 while True:
1559     best: tuple[float, int, int, list[dict[str, Any]]] | None = None
1560     for i in range(len(active)):
1561         for j in range(i + 1, len(active)):
1562             if len(active[i]["stops"]) != 1 or len(active[j]["stops"]) != 1:
1563                 continue
1564             if active[i]["stops"][0]["service_id"] == active[j]["stops"][0]["service_id"]:
1565                 continue
1566             individual = 0.0
1567             possible = True
1568             for spec in (active[i], active[j]):
1569                 values = [
1570                     route_timeline(data["dem"], data["nodes"]["001"], data["nodes"], g,
1571                                   boxes, spec["stops"], 0.0)
1572                     for g in data["transport_types"].values()
1573                 ]
1574                 values = [x for x in values if x is not None]
1575                 if not values:
1576                     possible = False
1577                     break
1578                 individual += min(x["energy_kwh"] for x in values)
1579             if not possible:
1580                 continue
1581             for order in ((active[i]["stops"][0], active[j]["stops"][0]),
1582                          [active[j]["stops"][0], active[i]["stops"][0]]):
1583                 merged = [
1584                     route_timeline(data["dem"], data["nodes"]["001"], data["nodes"], g,
1585                                   boxes, list(order), 0.0)
1586                     for g in data["transport_types"].values()
1587                 ]
1588                 merged = [x for x in merged if x is not None]
1589                 if not merged:
1590                     continue
1591                 saving = individual - min(x["energy_kwh"] for x in merged)
1592                 if saving > 1e-9 and (best is None or saving > best[0] + 1e-12):
1593                     best = (saving, i, j, list(order))
1594             if best is None:
1595                 break
1596     _, i, j, order = best
1597     active = [x for k, x in enumerate(active) if k not in (i, j)] + [
1598         {"stops": order, "kind": "soft"}
1599     ]
1600 return sorted(hard_specs + active, key=lambda x: _trip_spec_key(x, boxes))

```

代码清单5给出实体运输机和同型共享电池的事件排程核心。资源最早可用时刻共同决定任务开始，候选评分把硬迟到放在首位，随后比较软时限代价、交付、能耗和资源恢复时间。

Listing 5 运输机—电池组合生成与任务排程核心

```

1603 def _schedule_q2(data: dict[str, Any], specs: list[dict[str, Any]]) -> tuple[list[dict[str, Any]], list[dict[
    str, Any]]]:

```